

GRAPH NEURAL NETWORKS

SOMMAIRE

1-	Représentation d'un graphe	1
	1.1- Définition matricielle d'un graphe	
	1.2- Propriétés	
2-	Introduction aux GNN	2
	2.1- Utilisations	
	2.2- GCN	
	2.3- Application en classification	
	2.4- Modèles inductifs et transductifs	
	2.5- Exemple en classification de sommets	
	2.6- Couches d'un GNN	
	2.7- Prise en compte de l'information des arcs ou arêtes	
3-	Partie pratique	9

Comme leur nom l'indique, les Graph Neural Networks sont des réseaux de neurones qui traitent les graphes. Ce traitement pose trois problématiques :

- leur topologie est variable, et il est difficile de concevoir des réseaux qui soient à la fois suffisamment expressifs et capables de gérer cette variation.
- ils peuvent être de taille conséquente : un graphe représentant les connexions entre les utilisateurs d'un réseau social peut avoir plusieurs millions de sommets.
- il se peut qu'il n'y ait à disposition pour le problème à traiter qu'un seul graphe, de sorte que le protocole habituel d'entraînement avec de nombreux exemples de données et de test avec de nouvelles données n'est pas toujours possible.

1- REPRÉSENTATION D'UN GRAPHE

1.1- Définition matricielle d'un graphe

Soit $G = (V, E)$ un graphe à $N = |V|$ sommets et $M = |E|$ arcs (ou arêtes). Chaque sommet et chaque arc (ou arête) peut porter une information vectorielle (graphe pondéré). On choisit alors de représenter G par trois matrices A , X et E représentant respectivement la structure de G , les sommets et arcs (ou arêtes) :

1. $\mathbf{A}$ est la matrice d'adjacence (pour les arêtes) ou d'incidence (pour les arcs).
2. $\mathbf{X}$ est une matrice de taille $d \times N$, la i -ème colonne de $\mathbf{X}$ donnant les d informations portées par le sommet $i \in \llbracket 1, N \rrbracket$.
3. $\mathbf{E}$ est une matrice de taille $d_e \times M$, la i -ème colonne de $\mathbf{E}$ donnant les d_e informations portées par l'arc (ou l'arête) $i \in \llbracket 1, N \rrbracket$.

Dans un premier temps, on considérera uniquement le cas où $\mathbf{X}$ existe (seuls les sommets sont pondérés). On reviendra sur le cas de $\mathbf{E}$ dans la section 2.7-.

1.2- Propriétés

La matrice d'adjacence peut être utilisée pour trouver les voisins d'un sommet. Supposons que le i -ème sommet soit encodé sous la forme d'un vecteur colonne $\mathbf{X}_{\bullet,i}$ avec une seule entrée non nulle à la position i , fixée à un. En prémultipliant $\mathbf{X}_{\bullet,i}$ par la matrice $\mathbf{A}$, on calcule un vecteur avec des uns aux positions des voisins. En répétant cette procédure n fois, on accède aux voisins du sommet i accessibles en n étapes. Ainsi, le coefficient (l,c) de $\mathbf{A}^n$ contient le nombre de chaînes uniques de longueur n du sommet l au sommet c . A noter que ce n'est pas le nombre de chemins uniques puisqu'il inclut les chemins qui visitent le même sommet plus d'une fois.

Les sommets étant numérotés arbitrairement dans G , il est de plus possible de changer l'indexation de ces sommets, sans changer la structure de G , à l'aide d'une matrice de permutation $\mathbf{P}$. Les informations portées par les sommets sont alors résumées dans la matrice $\mathbf{XP}$, et la nouvelle matrice d'adjacence est donnée par $\mathbf{P}^T \mathbf{AP}$.

2- INTRODUCTION AUX GNN

Un GNN est un modèle qui prend les représentations des sommets $\mathbf{X}$ et la matrice d'adjacence $\mathbf{A}$ comme entrées et les fait passer par une série de K couches. Les représentations des sommets sont mises à jour à chaque couche pour créer des représentations intermédiaires cachées $\mathbf{H}_k$ avant de calculer les représentations de sortie $\mathbf{H}_K$, dont les colonnes comprennent des informations sur les sommets correspondants et leur contexte dans le G .

Les GNN peuvent être utilisés en exploitant la structure de G , les sommets et/ou les arcs ou arêtes.

2.1- Utilisations

2.1.1- En relation avec la structure de graphe

Le réseau attribue une étiquette ou estime une ou plusieurs valeurs à partir de l'ensemble du graphe, en exploitant à la fois la structure et la représentation des sommets. Par exemple, on peut vouloir prédire la température à laquelle un gaz (molécules représentées sous la forme d'un graphe) devient liquide (régression) ou si une molécule est toxique ou non (classification). Les représentations des sommets de sortie sont combinées (par exemple, en calculant la moyenne) et le vecteur résultant est mis en correspondance avec un vecteur de taille fixe par le biais d'une transformation linéaire ou d'un réseau de neurones. Pour la régression, le décalage entre le résultat et les valeurs de vérité terrain est calculé à l'aide de la fonction de perte moindres carrés. Pour la classification binaire, la sortie passe par une fonction sigmoïde et la perte est calculée à l'aide de l'entropie croisée binaire

$$P(y = 1 \mid \mathbf{X}, \mathbf{A}) = \sigma(\beta_K + \mathbf{w}_K \mathbf{H}_K \mathbf{1} / N)$$

où β_K et $\mathbf{w}_K^T \in \mathbb{R}^D$ sont des paramètres à apprendre. Multiplier à droite par le vecteur colonne $\mathbf{1}$ somme toutes les représentations, et diviser par N calcule la moyenne. La technique résultante est dite *mean pooling*.

2.1.2- En relation avec les sommets

Le réseau attribue une étiquette (classification) ou une ou plusieurs valeurs (régression) à chaque sommet du graphe, en utilisant à la fois la structure du graphe et les représentations des sommets. Par exemple, dans un graphe construit à partir d'un nuage de points 3D d'un avion, l'objectif peut être de classer les sommets selon qu'ils appartiennent aux ailes ou au fuselage (classification). En régression, on peut par exemple vouloir prédire le nombre de messages qu'un abonné d'un réseau social recevra. Les fonctions de perte sont définies de la même manière que pour les tâches au niveau du graphe, sauf que cette opération est effectuée indépendamment pour chaque sommet i :

$$P(y^{(i)} = 1 \mid \mathbf{X}, \mathbf{A}) = \sigma(\beta_K + \mathbf{w}_K \mathbf{h}_K^{(i)})$$

2.1.3- En relation avec les arcs ou arêtes

Le réseau prédit s'il doit y avoir ou non un arc (ou arête) entre les sommets i et j . Par exemple, dans le cadre d'un réseau social, le réseau peut prédire si deux personnes se connaissent et s'apprécient et suggérer qu'elles se connectent si c'est le cas. C'est une tâche de classification binaire pour laquelle la représentation des deux sommets doit être convertie en un nombre unique représentant la probabilité de présence de l'arc (ou arête). L'une des possibilités consiste à prendre le produit scalaire des représentations des sommets et à faire passer le résultat par une fonction sigmoïde pour calculer la probabilité.

2.2- GCN

Il existe de nombreux types de GNN, on se restreint ici aux réseaux convolutifs de graphes, ou GCN. Ces modèles sont convolutifs en ce sens qu'ils mettent à jour chaque sommet en agrégeant les informations provenant des sommets voisins. En tant que tels, ils induisent un biais inductif relationnel (une tendance à donner la priorité aux informations provenant des voisins). On suppose de plus que les convolutions s'opèrent dans le domaine spatial (utilisant la structure de G), plutôt que dans l'espace de Fourier (méthodes basées spectre). Chaque couche du GCN est une fonction $\mathbf{F}_{\phi}$, de paramètres ϕ , qui prend en entrée les représentations des sommets $\mathbf{X}$ et la matrice d'adjacence $\mathbf{A}$ de G et produit de nouvelles représentations des sommets :

$$\begin{aligned} \mathbf{H}_1 &= \mathbf{F}_{\phi_0}(\mathbf{X}, \mathbf{A}) \\ \mathbf{H}_2 &= \mathbf{F}_{\phi_1}(\mathbf{H}_1, \mathbf{A}) \\ &\dots \\ \mathbf{H}_K &= \mathbf{F}_{\phi_{K-1}}(\mathbf{H}_{K-1}, \mathbf{A}) \end{aligned}$$

les ϕ_k étant les paramètres du réseau entre la couche k et la couche $k + 1$

2.2.1- Equivariance et invariance

L'indexation des sommets dans le graphe étant arbitraire, il est indispensable que tout modèle respecte cette propriété. Chaque couche doit donc être équivariante¹ par rapport aux permutations des indices des sommets, soit pour toute permutation $\mathbf{P}$ et tout k :

$$\mathbf{H}_{k+1}\mathbf{P} = \mathbf{F}_{\phi_k}(\mathbf{H}_k\mathbf{P}, \mathbf{P}^T\mathbf{A}\mathbf{P})$$

Pour les tâches de classification des sommets et de prédiction des arcs ou arêtes, les résultats doivent également être équivariants pour les permutations des indices des sommets. Toutefois, pour les tâches en relation avec le graphe, la couche finale agrège les informations provenant de l'ensemble du graphe, de sorte que le résultat est invariant par rapport à l'ordre des sommets.

1. Une fonction f est équivariante pour une transformation t si pour tout x , $f(t(x)) = t(f(x))$

2.2.2- Partage des paramètres

Dans les réseaux convolutifs, des couches convolutives sont utilisées, qui traitent chaque position de l'image de manière identique. Cela permet de réduire le nombre de paramètres et d'introduire un biais inductif qui force le modèle à traiter chaque partie de l'image de la même manière. Le même argument peut être avancé pour les sommets d'un graphe. On pourrait apprendre un modèle avec des paramètres distincts associés à chaque sommet. Cependant, le réseau doit maintenant apprendre indépendamment la signification des connexions dans le graphe à chaque position, et l'apprentissage nécessiterait de nombreux graphes ayant la même topologie. Il est plus judicieux de construire un modèle qui utilise les mêmes paramètres à chaque sommet, réduisant ainsi le nombre de paramètres et partageant ce que le réseau apprend à chaque sommet sur l'ensemble du graphe.

On peut modéliser une convolution (qui met à jour une variable en prenant une somme pondérée des informations provenant de ses voisins) comme le fait que chaque voisin envoie un message à la variable d'intérêt, qui agrège ces messages pour former la mise à jour. Dans le cas des images, les voisins sont les pixels d'une région carrée de taille fixe autour de la position actuelle, de sorte que les relations spatiales à chaque position sont les mêmes. Dans un graphe, chaque sommet peut avoir un nombre différent de voisins et il n'existe a priori pas de relation privilégiée entre sommets : il n'y a aucune raison de pondérer favorablement (ou pas) les informations provenant d'un sommet particulier.

2.2.3- Exemple

La figure 2-1 présente un exemple simple de GCN.

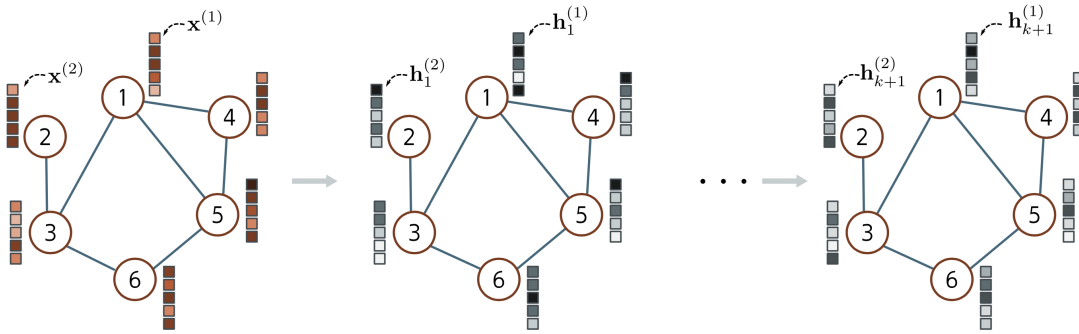


FIGURE 2-1 – Exemple de GCN

- à gauche le graphe G initial, les colonnes de $\mathbf{X}$ étant reportées à côté des sommets correspondants.
- au milieu, chaque sommet de la première couche cachée est mis à jour en :
 1. agrégeant les sommets voisins d'un sommet i en un unique vecteur : $\mathbf{f}_1(i) = \sum_{j \text{ voisin de } i} \mathbf{h}_1(j)$
 2. appliquant pour tout i une transformation linéaire $\mathbf{L}$ au sommet initial $\mathbf{x}^i$ et aux sommets agrégés et en ajoutant un biais β_0 : $\beta_0 + \mathbf{L}\mathbf{x}^i + \mathbf{L}\mathbf{f}_1(i)$
 3. appliquant une fonction non linéaire g au résultat précédent : $\mathbf{h}_1^i = g(\beta_0 + \mathbf{L}\mathbf{x}^i + \mathbf{L}\mathbf{f}_1(i))$
- à droite, le processus répété pour toute couche k :

$$(\forall i) \quad \mathbf{h}_{k+1}^i = g \left(\beta_k + \mathbf{L}_k \mathbf{h}_k^i + \mathbf{L}_k \left(\sum_{j \text{ voisin de } i} \mathbf{h}_k(j) \right) \right)$$

On peut écrire ce processus de manière matricielle : Si $\mathbf{H}_k \in \mathcal{M}_{D,N}(\mathbb{R})$ est la matrice dont les colonnes sont les représentations des sommets, alors

$$\mathbf{H}_{k+1} = g \left(\beta_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k + \mathbf{L}_k \mathbf{H}_k \mathbf{A} \right) = g \left(\beta_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k (\mathbf{A} + \mathbf{I}) \right)$$

où g est appliquée point à point sur les éléments de la matrice argument. On remarque que la couche $k + 1$ est bien équivariante à la permutation de la numérotation des sommets, utilise la structure du graphe ($\mathbf{A}$) pour produire un biais inductif et partage les paramètres sur tout le graphe.

2.3- Application en classification

Pour l'exemple, on s'intéresse à un problème de classification binaire. On modélise une molécule comme un graphe, sont les sommets sont les atomes. La matrice $\mathbf{A}$ donne les liaisons entre les atomes, et la matrice $\mathbf{X}$ donne le nom de l'atome : si la table périodique des éléments comporte D atomes, le sommet (l'atome) i est un vecteur de $\{0, 1\}^D$, où la seule composante qui vaille 1 est celle qui identifie le type de l'atome. On s'intéresse alors de savoir si une molécule donnée est toxique ($y = 1$) ou pas ($y = 0$).

Les équations du réseau sont alors :

$$\forall k \in \llbracket 0, K - 1 \rrbracket \quad \mathbf{H}_{k+1} = g \left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k (\mathbf{A} + \mathbf{I}) \right)$$

et

$$f(\mathbf{X}, \mathbf{A}, \boldsymbol{\phi}) = P(y = 1 \mid \mathbf{X}, \mathbf{A}) = \sigma(\boldsymbol{\beta}_K + \mathbf{w}_K \mathbf{H}_K \mathbf{1} / N)$$

où $\boldsymbol{\phi} = (\boldsymbol{\beta}_k, \mathbf{L}_k)_{k \in \llbracket 0, K \rrbracket}$ sont les paramètres du réseau à apprendre et σ la fonction sigmoïde.

Étant donnés n exemples d'entraînement $(\mathbf{X}_i, \mathbf{A}_i, y_i)_{i \in \llbracket 1, n \rrbracket}$, $\boldsymbol{\phi}$ peut être classiquement appris par minimisation de l'entropie croisée binaire sur des batchs d'exemples. Si dans les MLP et les CNN, les entrées sont de taille identique (et donc les exemples sont concaténés en un tenseur de dimension supérieure pour un entraînement efficace par GPU ou TPU), les graphes de la base d'entraînement ont très probablement un nombre de sommets N et une dimension de l'espace de représentation D différents, ce qui rend cette concaténation impossible. Une astuce simple permet cependant de traiter l'ensemble du batch en parallèle. Les graphes du batch sont traités comme des composantes disjointes d'un seul grand graphe. Le réseau peut alors être exécuté comme une instance unique des équations de réseau. La mise en commun des moyennes est effectuée uniquement sur les graphes individuels afin d'obtenir une représentation unique par graphe qui peut être introduite dans la fonction de perte.

2.4- Modèles inductifs et transductifs

Jusqu'à présent, tous les modèles présentés dans ce cours ont été inductifs : on exploite un ensemble de données étiquetées pour apprendre la relation entre les entrées et les sorties. On l'applique ensuite à de nouvelles données de test. En d'autres termes, on apprend la règle qui associe les entrées aux sorties, puis on l'applique ailleurs.

En revanche, un modèle transductif (ou apprentissage semi-supervisé) prend en compte les données étiquetées et non étiquetées en même temps. Il ne produit pas de règle, mais simplement une étiquette pour les sorties inconnues. Il présente l'avantage de pouvoir utiliser des modèles sur des données non étiquetées pour prendre ses décisions, mais nécessite un nouvel entraînement du modèle lorsque des données non étiquetées supplémentaires sont ajoutées.

Les deux types de problèmes sont couramment rencontrés pour les graphes. Parfois, on dispose de nombreux graphes étiquetés et on apprend une correspondance entre le graphe et les étiquettes. D'autres fois, il arrive qu'il n'y ait à disposition qu'un seul graphe de très grande dimension et dans ce cas, les données d'apprentissage et de test sont nécessairement connectées.

Les utilisations des GNN en relation avec la structure des graphes ne se produisent que dans le cadre inductif où il existe des graphes d'apprentissage et de test. Toutefois, les utilisations en relation avec les sommets et les tâches de prédiction des arcs ou arêtes peuvent se produire dans l'un ou l'autre cadre. Dans le cas transductif, la fonction de perte minimise le décalage entre la sortie du modèle et la vérité lorsqu'elle est connue. Les nouvelles prédictions sont calculées en exécutant la passe avant et en récupérant les résultats lorsque la vérité est inconnue.

2.5- Exemple en classification de sommets

On s'intéresse ici à un problème de classification binaire des sommets dans un cadre transductif. Le graphe considéré comporte des millions de sommets, certains ayant des étiquettes binaires y_i . L'objectif est alors d'étiqueter les sommets non étiquetés restants. Le réseau est le même que dans l'exemple 2.3- avec une couche finale différente qui produit un vecteur de sortie de taille $1 \times N$:

$$f(\mathbf{X}, \mathbf{A}, \boldsymbol{\phi}) = \sigma(\boldsymbol{\beta}_K \mathbf{1}^T + \mathbf{w}_K \mathbf{H}_K)$$

la fonction σ agissant point à point. On trouve $\boldsymbol{\phi}$ par minimisation de l'entropie croisée binaire, mais seulement à partir des valeurs des sommets pour lesquels les étiquettes y_i sont connues.

L'entraînement de ce réseau pose deux problèmes. Tout d'abord, il est difficile d'entraîner un réseau de cette taille, ne serait-ce que parce qu'il faut stocker les représentations des sommets à chaque couche du réseau dans la passe avant. Cela implique à la fois le stockage et le traitement d'une structure plusieurs fois plus grande que le graphe entier. De plus, n'ayant qu'un seul graphe à disposition, la descente de gradient (ou tout autre algorithme d'optimisation) sur batch est impossible, puisqu'un seul objet constitue la base d'entraînement.

Pour répondre à ce second problème, on choisit un sous-ensemble aléatoire de sommets étiquetés à chaque étape de l'entraînement. Chaque sommet dépend de ses voisins dans la couche précédente. Ces derniers dépendent à leur tour de leurs voisins de la couche précédente, de sorte que chaque sommet possède l'équivalent d'un champ réceptif comme dans les CNN. La taille du champ réceptif est appelée voisinage à k sauts. On peut donc effectuer une étape de descente de gradient en utilisant le graphe qui forme l'union des voisinages de k -sauts des sommets du batch. Les entrées restantes ne contribuent pas. S'il y a de nombreuses couches et que le graphe est fortement connecté, chaque sommet d'entrée peut se trouver dans le champ réceptif de chaque sortie, ce qui ne réduit pas du tout la taille du graphe : c'est le problème de l'expansion du graphe. Deux approches s'attaquent à ce problème : l'échantillonnage du voisinage et le partitionnement du graphe.

2.5.1- Échantillonnage du voisinage

Le graphe complet est échantillonné, ce qui réduit les connexions à chaque couche du réseau. Par exemple, on peut commencer par les sommets du batch et échantillonner aléatoirement un nombre fixe de leurs voisins dans la couche précédente. Ensuite, on échantillonne au hasard un nombre fixe de leurs voisins dans la couche précédente, et ainsi de suite. La taille du graphe augmente toujours à chaque couche, mais de manière beaucoup plus contrôlée. Cette opération est renouvelée pour chaque batch, de sorte que les voisins contributeurs diffèrent même si le même batch est tiré deux fois. Cette technique rappelle celle du dropout et ajoute une certaine régularisation.

2.5.2- Partitionnement du graphe

On peut également partitionner le graphe original en sous-ensembles de sommets disjoints, et construire des graphes plus petits qui ne sont pas connectés les uns aux autres avant le traitement. Il existe des algorithmes standards pour choisir ces sous-ensembles afin de maximiser le nombre de liens internes. Ces petits graphes peuvent chacun être traités comme des batchs, ou un sous-ensemble aléatoire d'entre eux peut être combiné pour former un batch (en rétablissant toutes les arêtes entre eux à partir du graphe d'origine).

Utilisant l'une de ces deux approches, il est alors possible d'entraîner les paramètres du réseau de la même manière que pour le cadre inductif, en divisant les sommets étiquetés en ensembles d'entraînement, de test et de validation comme souhaité. Pour effectuer l'inférence, on calcule les prédictions pour les sommets inconnus sur la base de leur voisinage de k -sauts. Contrairement à l'entraînement, il n'est pas nécessaire de stocker les représentations intermédiaires, ce qui rend l'utilisation de la mémoire beaucoup plus efficace.

2.6- Couches d'un GNN

les sections précédentes combinaient les sommets adjacents par sommation, en multipliant $\mathbf{H}$ par $\mathbf{A} + \mathbf{I}$. Dans la suite de ce paragraphe, on présente des alternatives à cette approche.

2.6.1- Amélioration de la diagonale

La mise à jour proposée jusqu'à lors

$$\forall k \in \llbracket 0, K-1 \rrbracket \mathbf{H}_{k+1} = g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k (\mathbf{A} + \mathbf{I})\right)$$

peut être modifiée en

$$\forall k \in \llbracket 0, K-1 \rrbracket \mathbf{H}_{k+1} = g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k (\mathbf{A} + (1 + \varepsilon_k) \mathbf{I})\right)$$

où ε_k est appris, ou en

$$\begin{aligned} \forall k \in \llbracket 0, K-1 \rrbracket \mathbf{H}_{k+1} &= g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k \mathbf{A} + \boldsymbol{\Psi}_k \mathbf{H}_k\right) \\ &= g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \begin{pmatrix} \mathbf{L}_k & \boldsymbol{\Psi}_k \end{pmatrix} \begin{pmatrix} \mathbf{H}_k \mathbf{A} \\ \mathbf{H}_k \end{pmatrix}\right) \\ &= g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}'_k \begin{pmatrix} \mathbf{H}_k \mathbf{A} \\ \mathbf{H}_k \end{pmatrix}\right) \end{aligned}$$

où $\mathbf{L}'_k = \begin{pmatrix} \mathbf{L}_k & \boldsymbol{\Psi}_k \end{pmatrix}$ permet d'appliquer une transformation linéaire différente au sommet courant.

2.6.2- Connexions résiduelles

Avec les connexions résiduelles, la représentation agrégée des voisins est transformée et passe par la fonction d'activation avant d'être additionnée ou concaténée avec le sommet actuel :

$$\mathbf{H}_{k+1} = \begin{pmatrix} g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k \mathbf{A}\right) \\ \mathbf{H}_k \end{pmatrix}$$

2.6.3- Agrégation moyenne

Les méthodes précédentes regroupent les voisins en additionnant les représentation des sommets. Cependant, il est possible de combiner différemment ces représentations. Parfois, il est préférable de prendre la moyenne des voisins plutôt que la somme. Cette méthode peut s'avérer plus performante si les informations de représentation sont plus importantes et les informations structurelles moins, car la part de contribution du voisinage ne dépend pas du nombre de voisins :

$$\mathbf{f}(i) = \frac{1}{|\mathcal{V}'_i|} \sum_{j \in \mathcal{V}'_i} \mathbf{h}(j)$$

où $\mathcal{V}'_i$ désigne l'ensemble des voisins du sommet i . En notation matricielle, si $\mathbf{D}$ est la matrice diagonale des degrés alors

$$\forall k \in \llbracket 0, K-1 \rrbracket \mathbf{H}_{k+1} = g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k (\mathbf{A} \mathbf{D}^{-1} + \mathbf{I})\right)$$

2.6.4- Normalisation de Kipf

Ici

$$\mathbf{f}(i) = \sum_{j \in \mathcal{V}'_i} \frac{\mathbf{h}(j)}{\sqrt{|\mathcal{V}'_i| |\mathcal{V}'_j|}}$$

l'information provenant des sommets ayant un grand nombre de voisins devant être revue à la baisse (il existe un grand nombre d'arcs qui fournissent moins d'information unique). En notation matricielle, cette normalisation s'écrit

$$\forall k \in \llbracket 0, K-1 \rrbracket \mathbf{H}_{k+1} = g\left(\boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k (\mathbf{D}^{-\frac{1}{2}} \mathbf{A} \mathbf{D}^{-\frac{1}{2}} + \mathbf{I})\right)$$

2.6.5- Agrégation par max pooling

Comme dans le cas des CNN, on peut envisager d'agréger par le max, qui s'effectue alors composante par composante.

$$\mathbf{f}(i) = \max_{j \in \mathcal{V}_i} \mathbf{h}(j)$$

2.6.6- Agrégation par attention

Les méthodes d'agrégation examinées jusqu'à présent pondèrent la contribution des voisins de manière égale ou d'une manière qui dépend de la topologie du graphe. Inversement, dans les couches d'attention de graphe, les poids dépendent des données aux sommets. Une transformation linéaire est appliquée aux représentations des sommets

$$\forall k \in \llbracket 0, K-1 \rrbracket \quad \mathbf{H}'_k = \boldsymbol{\beta}_k \mathbf{1}^T + \mathbf{L}_k \mathbf{H}_k$$

La similarité s_{ij} entre les représentations transformées $\mathbf{h}'_i, \mathbf{h}'_j$ des sommets i et j est calculée en concaténant les paires, en effectuant un produit scalaire avec un vecteur colonne $\boldsymbol{\phi}_k$ de paramètres appris et en appliquant une fonction d'activation

$$s_{ij} = g \left(\boldsymbol{\phi}_k^T \begin{pmatrix} \mathbf{h}'_i \\ \mathbf{h}'_j \end{pmatrix} \right)$$

Les similarités sont stockées dans une matrice $\mathbf{S}$. Comme pour les mécanismes d'attention, les poids doivent être positifs et de somme 1, mais pour un sommet donné, seuls lui et ses voisins doivent contribuer. On effectue donc l'opération

$$\forall k \in \llbracket 0, K-1 \rrbracket \quad \mathbf{H}_{k+1} = g(\mathbf{H}'_k \cdot \text{Softmax}(\mathbf{S}, \mathbf{A} + \mathbf{I}))$$

la fonction $\text{Softmax}(\bullet, \bullet)$ calcule les valeurs d'attention en appliquant l'opération softmax séparément à chaque colonne de son premier argument $\mathbf{S}$, mais seulement après avoir fixé à $-\infty$ les valeurs pour lesquelles le deuxième argument $\mathbf{A} + \mathbf{I}$ est égal à zéro, de sorte qu'elles ne contribuent pas. Cela garantit que l'attention accordée aux sommets non voisins est nulle.

2.7- Prise en compte de l'information des arcs ou arêtes

Les paragraphes précédents ont abordé le traitement des représentations des sommets. Ceux-ci évoluent au fur et à mesure qu'ils sont transmis dans le réseau, de sorte qu'à la fin ils représentent à la fois le sommet et son contexte dans le graphe. On considère maintenant le cas où l'information est associée aux arêtes du graphe.

Il est facile d'adapter le mécanisme de représentation précédent pour traiter la représentation des arêtes à l'aide du graphe des arêtes (ou graphe adjoint). Il s'agit d'un graphe complémentaire, dans lequel chaque arête du graphe original devient un sommet, et chaque paire d'arêtes ayant un sommet commun dans le graphe original crée une arête dans le nouveau graphe. En général, un graphe peut être reconstruit à partir de son graphe d'arêtes, de sorte qu'il est possible de passer d'une représentation à l'autre.

Une fois le graphe d'arêtes construit, on utilise les mêmes techniques, en agrégeant les informations de chaque nouveau sommet à partir de ses voisins et en les combinant avec la représentation actuelle. Lorsque les représentations des sommets et d'arêtes sont tous deux présents, on peut passer d'un graphe à l'autre. Il existe donc quatre mises à jour possibles (les sommets mettent à jour les sommets, les sommets mettent à jour les arêtes, les arêtes mettent à jour les sommets et les arêtes mettent à jour les arêtes), qui peuvent être alternées à volonté ou, moyennant des modifications mineures, les sommets peuvent être mis à jour simultanément à partir des sommets et des arêtes.

3- PARTIE PRATIQUE

Pour bien comprendre la structure d'un GNN, on propose de ne pas utiliser de librairie dédiée (type [Spectral](#), [StellarGraph](#) ou encore [GraphNets](#), mais plutôt de l'implémenter directement à partir de Tensorflow et Keras.

BIBLIOGRAPHIE